

PARALLEL LOGISTIC REGRESSION FOR WEATHER FORECASTING USING HYBRID MPI-BASED ARCHITECTURES

Bhagyouday Nagarwad
Roll No: AM25M005
Department of Applied Mechanics
Indian Institute of Technology Madras
Chennai, India

ABSTRACT

The rapid growth of large-scale meteorological datasets has created a demand for efficient computational methods capable of delivering timely and accurate predictions. Traditional sequential machine learning algorithms are often insufficient for processing such large volumes of data due to high computational complexity and memory constraints.

In this work, a Logistic Regression model is developed and parallelized for rainfall prediction using real-world weather datasets consisting of features such as temperature, humidity, atmospheric pressure, and wind speed. Two hybrid parallel approaches are implemented: MPI + OpenMP for CPU-based shared and distributed memory parallelism, and MPI + OpenACC for GPU-accelerated distributed computing.

The dataset is partitioned across processes, and training is performed using gradient descent with distributed gradient computation and global aggregation using MPI communication primitives. Data preprocessing includes normalization and class imbalance handling using weighted loss functions.

Performance analysis demonstrates significant reduction in execution time compared to the serial implementation. The MPI + OpenMP model achieves efficient CPU utilization, while MPI + OpenACC provides further acceleration using GPUs. However, scalability is limited beyond a certain number of processes due to communication overhead.

This work highlights the effectiveness of hybrid parallel computing techniques in accelerating machine learning algorithms for large-scale data and demonstrates their applicability in time-critical domains such as weather forecasting.

NOMENCLATURE

X	Feature matrix
y	Target variable
n	Number of samples
m	Number of features
w	Model weights
b	Bias term
MPI	Message Passing Interface
OpenMP	Shared-memory model
OpenACC	GPU acceleration model

INTRODUCTION

Weather forecasting is an important scientific and engineering problem because it supports decision-making in agriculture, transportation, disaster management, water resources, and urban planning. With the increasing availability of large-scale meteorological datasets, data-driven models are becoming useful tools for rainfall prediction. However, training machine learning models on millions of samples can be computationally expensive when implemented sequentially.

Logistic Regression is a simple and interpretable classification algorithm that is widely used for binary prediction problems. In this project, it is used for rainfall classification, where the output is represented as either rain or no rain. The model takes meteorological parameters such as temperature, humidity, pressure, and wind-related quantities as input features and predicts the probability of rainfall.

Although Logistic Regression is computationally simpler than deep learning models, training on large datasets still requires repeated operations over all samples for many epochs. Each training epoch involves computing predictions, errors, gradients, and parameter updates. As the dataset size increases, the execution time of the serial implementation becomes significant. Therefore, parallel computing techniques are required to improve training efficiency.

In this work, two hybrid parallel implementations are developed and analyzed. The first implementation combines MPI and OpenMP, where MPI distributes the dataset among multiple processes and OpenMP accelerates computations within each process using shared-memory parallelism. The second implementation combines MPI and OpenACC, where MPI handles distributed data processing and OpenACC offloads computationally intensive operations to the GPU.

OBJECTIVES

The main objectives of this project are:

- To develop a serial Logistic Regression model for binary rainfall classification using large-scale weather data.
- To preprocess the dataset using feature normalization and class imbalance handling.
- To identify the computationally expensive parts of the serial algorithm, especially gradient computation.
- To parallelize the training loop using OpenMP for shared-memory CPU parallelism.
- To extend the OpenMP-based implementation by adding MPI for distributed-memory data parallelism.
- To develop and compare MPI + OpenMP and MPI + OpenACC versions for CPU and GPU-based acceleration.
- To evaluate model performance using accuracy, precision, recall, F1-score, and confusion matrix.
- To analyze parallel performance using execution time, speedup, efficiency, and scalability.

PROBLEM STATEMENT

Large-scale weather datasets contain millions of samples and several meteorological features. Training a machine learning model on such data using a sequential algorithm requires significant computational time because every epoch involves repeated prediction, error calculation, gradient computation, and parameter updates over the complete dataset. The problem addressed in this project is to reduce the training time of a Logistic Regression model for rainfall classification while maintaining prediction accuracy. The model classifies weather samples into two classes: rain and no rain, using input features such as temperature, humidity, pressure, and wind-related quantities.

A serial implementation is first developed as the baseline model. OpenMP is then introduced to parallelize the computationally expensive loops on a shared-memory CPU

system. After validating the OpenMP version, MPI is added to distribute the dataset across multiple processes, allowing each process to compute local gradients independently. These gradients are combined using MPI collective communication operations such as MPI_Allreduce.

In addition to the MPI + OpenMP version, an MPI + OpenACC implementation is also explored to study GPU-based acceleration. The main challenge is to compare these parallel strategies in terms of execution time, speedup, efficiency, scalability, and model performance.

MATHEMATICAL FORMULATION

Logistic Regression is used in this work for binary rainfall classification, where the output class represents either rain or no rain. Each input sample consists of meteorological features such as temperature, humidity, pressure, and wind speed. For a dataset with n samples and m features, the input feature matrix and target vector are represented as:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

where $y_i \in \{0, 1\}$, with 0 representing no rain and 1 representing rain.

Feature Normalization

Before training, each feature is normalized using standardization:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j + \epsilon}$$

where μ_j is the mean of the j^{th} feature, σ_j is the standard deviation of the j^{th} feature, and ϵ is a small value used to avoid division by zero.

Logistic Regression Model

For each input sample x_i , the linear model is computed as:

$$z_i = w^T x_i + b$$

where w is the weight vector and b is the bias term. The sigmoid activation function converts this linear output into a probability:

$$\hat{y}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

Here, $\hat{y}_i$ represents the predicted probability of rainfall.

The final predicted class is obtained using a threshold value:

$$\text{Prediction} = \begin{cases} 1, & \hat{y}_i > 0.4 \\ 0, & \hat{y}_i \leq 0.4 \end{cases}$$

In this project, a threshold of 0.4 is used to improve rainfall detection, especially because the dataset contains class imbalance.

Class Imbalance Handling

Since rainfall events may occur less frequently than no-rain events, class weights are used to balance the contribution of both classes during training. The weights are computed as:

$$w_1 = \frac{n}{2N_1}$$

$$w_0 = \frac{n}{2N_0}$$

where N_1 is the number of rain samples, N_0 is the number of no-rain samples, and n is the total number of samples.

The weighted error term for each sample is defined as:

$$e_i = \begin{cases} w_1(\hat{y}_i - y_i), & y_i = 1 \\ w_0(\hat{y}_i - y_i), & y_i = 0 \end{cases}$$

Gradient Descent Optimization

The model parameters are updated using gradient descent. The gradients of the weights and bias are computed as:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n e_i x_{ij}$$

$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n e_i$$

The weight and bias updates are:

$$w_j = w_j - \alpha \frac{\partial J}{\partial w_j}$$

$$b = b - \alpha \frac{\partial J}{\partial b}$$

where α is the learning rate. In this implementation, the learning rate is set to 0.05 and the model is trained for 1500 epochs.

Evaluation Metrics

The trained model is evaluated using the confusion matrix values: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). The evaluation metrics are defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

These metrics are used to evaluate both the overall prediction accuracy and the ability of the model to correctly detect rainfall events.

METHODOLOGY

The overall methodology adopted in this work consists of data preprocessing, serial model development, parallelization, and performance evaluation. The workflow followed during the implementation is summarized below.

Dataset Preparation

The weather dataset used in this work contains multiple meteorological parameters related to rainfall prediction. The dataset is read from a CSV file and separated into input features and target labels. The target variable is binary, representing rain or no rain conditions.

To improve numerical stability and model convergence, feature normalization is performed using mean and standard deviation scaling. Since the dataset is imbalanced, weighted loss handling is introduced by assigning separate weights to rain and no-rain samples.

Serial Logistic Regression Model

A serial Logistic Regression model is first implemented as the baseline version. The sigmoid activation function is used to convert the linear output into probability values between 0 and 1.

The model parameters are trained using gradient descent optimization. During each

epoch, predictions are computed for all samples, errors are evaluated, gradients are accumulated, and model parameters are updated. The serial implementation is also used to verify correctness before introducing parallel computing techniques.

OpenMP Parallelization

After validating the serial implementation, OpenMP is introduced to parallelize computationally expensive loops in the training phase. Shared-memory parallelism is applied mainly during gradient computation and prediction operations. The OpenMP implementation allows multiple CPU threads to execute loop iterations simultaneously, thereby reducing execution time on multi-core systems.

MPI-Based Parallelization

To further improve scalability, MPI is added to distribute the dataset across multiple processes. Each MPI process receives a subset of the dataset and computes local gradients independently.

After local gradient computation, MPI collective communication operations such as MPI.Allreduce are used to combine local gradients into global gradients for parameter updates.

This distributed-memory approach reduces the workload per process and enables parallel execution across multiple processing units.

MPI + OpenACC Hybrid Model

In addition to CPU-based parallelization, GPU acceleration is explored using OpenACC directives. The computationally intensive loops are offloaded to the GPU for parallel execution.

The MPI + OpenACC implementation combines distributed-memory parallelism with GPU acceleration, enabling faster gradient computation for large-scale datasets.

Performance Evaluation

The performance of the developed models is evaluated using both machine learning metrics and parallel performance metrics.

Classification performance is measured using:

- Accuracy
- Precision
- Recall
- F1-score

- Confusion Matrix

Parallel performance is analyzed using:

- Execution Time
- Speedup
- Efficiency
- Scalability

ALGORITHM

The training procedure of the Logistic Regression model used in this work is summarized below.

1. Read the weather dataset from the input CSV file.
2. Separate the input features and target labels.
3. Compute the number of rain and no-rain samples.
4. Calculate class weights to handle data imbalance.
5. Normalize each feature using mean and standard deviation.
6. Initialize model parameters: weight vector w and bias b .
7. Set the learning rate, number of epochs, and classification threshold.
8. For each epoch:
 - (a) Set gradients $\frac{\partial J}{\partial w}$ and $\frac{\partial J}{\partial b}$ to zero.
 - (b) For each training sample:
 - i. Compute the linear output:

$$z_i = w^T x_i + b$$

- ii. Compute predicted probability using sigmoid:

$$\hat{y}_i = \frac{1}{1 + e^{-z_i}}$$

- iii. Compute weighted error term.
- iv. Accumulate gradients for all weights and bias.
- (c) Update weights and bias using gradient descent.
9. After training, compute predictions on the dataset.
10. Form the confusion matrix using TP, TN, FP, and FN.
11. Compute performance metrics: accuracy, precision, recall, and F1-score.
12. Measure total execution time for performance analysis.

Parallel Extension

For the parallel implementations, the above algorithm is extended as follows:

- In the OpenMP version, the computationally expensive loops are parallelized using shared-memory threads.

- In the MPI version, the dataset is divided among multiple processes.
- Each MPI process computes local gradients using its assigned data.
- The local gradients are combined using MPI collective communication such as MPI.Allreduce.
- In the MPI + OpenACC version, gradient computation loops are offloaded to the GPU for acceleration.

FLOWCHART

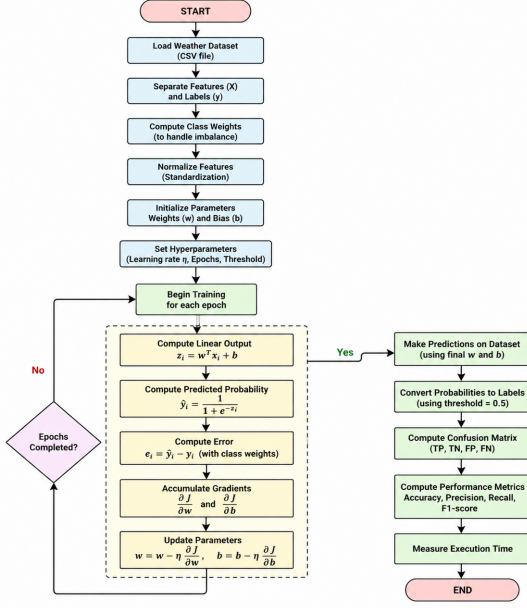


Figure 1: Flowchart of the Logistic Regression training and evaluation process.

IMPLEMENTATION

The implementation of the proposed rain-fall prediction model was carried out progressively, starting from a serial Logistic Regression model and extending towards hybrid parallel implementations using OpenMP, MPI, and OpenACC. The entire project was implemented in C++.

Serial Implementation

The serial implementation serves as the baseline model for correctness verification and performance comparison. The weather dataset is first loaded from a CSV file and divided into input features and target labels. Feature normalization is performed using mean and standard deviation scaling to improve convergence during training. The Logistic Regression model is trained using gradient descent optimization. During

each training epoch, the following operations are performed sequentially:

- Computation of the linear output
- Evaluation of sigmoid probabilities
- Error calculation
- Gradient accumulation
- Weight and bias updates

The serial implementation processes the complete dataset on a single CPU core and is used to measure the baseline execution time.

OpenMP Implementation

After validating the serial model, OpenMP parallelization was introduced to accelerate the computationally intensive loops. Shared-memory parallelism was applied mainly during gradient computation and prediction evaluation.

OpenMP directives were added to parallelize loops across multiple CPU threads. Since different threads update common gradient variables, synchronization and reduction operations were carefully handled to maintain correctness.

The OpenMP implementation reduced execution time significantly compared to the serial version by utilizing multiple CPU cores simultaneously.

MPI Implementation

To further improve scalability, MPI was introduced for distributed-memory parallelism. The dataset was partitioned across multiple MPI processes so that each process handled only a subset of the total samples.

Each MPI process independently computed local gradients using its assigned data partition. After local computation, global gradients were obtained using MPI collective communication operations such as MPI.Allreduce.

The MPI implementation enabled parallel execution across multiple processes and reduced the workload handled by each process. The execution time decreased as the number of MPI processes increased.

MPI + OpenMP Hybrid Implementation

A hybrid MPI + OpenMP model was also implemented by combining distributed-memory and shared-memory parallelism. MPI was

used to distribute the dataset among processes, while OpenMP parallelized computations within each process.

This hybrid strategy improved CPU utilization by combining inter-process parallelism with intra-process multi-threading. The implementation demonstrated better scalability than using OpenMP alone.

MPI + OpenACC Implementation

GPU acceleration was explored using OpenACC directives. The computationally expensive loops involved in prediction and gradient computation were offloaded to the GPU.

In the MPI + OpenACC implementation, MPI handled distributed-memory parallelism, while OpenACC accelerated local computations on the GPU. This hybrid approach utilized both CPU and GPU resources for large-scale machine learning computation. The NVIDIA HPC SDK compiler was used to compile the OpenACC version. GPU acceleration provided additional performance improvement for large datasets.

Performance Measurement

Execution time was measured using high-resolution timers from the C++ chrono library. Parallel performance was analyzed using execution time, speedup, efficiency, and scalability for different numbers of MPI processes.

The final implementation was tested on large-scale weather datasets containing millions of samples to evaluate both computational performance and prediction accuracy.

Results and Discussion

Dataset Description

The dataset used in this work consists of approximately 3 million weather records generated using Python. The data generation process was designed by considering realistic meteorological ranges corresponding to Indian weather conditions, specifically focusing on the Chennai region. Each sample contains six important atmospheric parameters commonly associated with rainfall prediction. The input features used in the dataset are listed below:

- Temperature

- Humidity
- Visibility
- Cloud Cover
- Wind Speed
- Atmospheric Pressure

The target variable is binary in nature, where class 0 represents no-rain conditions and class 1 represents rainfall occurrence. To maintain realistic weather distribution while ensuring sufficient rainfall samples during training, the dataset was balanced using approximately 75% no-rain samples and 25% rain samples.

The generated dataset contains nearly 2.56 million samples after preprocessing and cleaning operations. Before training, feature normalization was applied in order to improve numerical stability and accelerate convergence of the Logistic Regression model. Weighted error computation was also incorporated to handle class imbalance more effectively during training.

Experimental Setup

All serial and parallel implementations were executed on the same computing platform to ensure fair performance comparison. The experiments were performed on a system equipped with a 12th Gen Intel(R) Core(TM) i7-12700H processor and 16 GB SK Hynix DDR4-3200 MHz RAM.

GPU acceleration experiments were carried out using an NVIDIA GeForce RTX 3050 Laptop GPU with 4 GB VRAM support. CUDA version 13.1 together with WSL2 support was used for enabling GPU-based OpenACC execution.

The operating system used for experimentation was Windows 11 (64-bit) together with Windows Subsystem for Linux (WSL2). All implementations were developed in C++ using different parallel programming models including OpenMP, MPI, and OpenACC.

The GNU compiler suite (g++), MPI compiler (mpicxx), and NVIDIA HPC SDK compiler were used for compiling the serial and parallel implementations. OpenMP was used for shared-memory parallelization, MPI was used for distributed-memory communication, and OpenACC was used for GPU acceleration.

Serial Performance Analysis

The serial version of the Logistic Regression model was executed using 1000 training

epochs on the complete weather dataset containing approximately 2.56 million samples. The execution time obtained for the serial computation was approximately 134.19 seconds.

The serial implementation achieved an overall classification accuracy of approximately 82.05%. The obtained precision, recall, and F1-score values indicate stable rainfall classification performance for the generated weather dataset.

The large execution time observed in the serial execution demonstrates the computational cost associated with processing large-scale datasets using sequential computation. Since all prediction, error computation, and gradient update operations are executed on a single CPU core, the training process becomes computationally intensive as the dataset size increases.

The serial execution results were used as the baseline reference for evaluating the performance improvement obtained through OpenMP, MPI + OpenMP, and MPI + OpenACC parallel implementations.

OpenMP Execution Time Analysis

The OpenMP implementation was tested by varying the number of threads while keeping the dataset size and number of epochs fixed. The purpose of this experiment was to study the effect of shared-memory parallelism on the execution time of the Logistic Regression training process.

The serial execution time for 1000 epochs was approximately 134.19 seconds. When OpenMP was used, the execution time decreased as the number of threads increased. With 16 threads, the execution time reduced to approximately 19.41 seconds. This shows that the computationally intensive loops in the Logistic Regression training process were effectively parallelized using OpenMP.

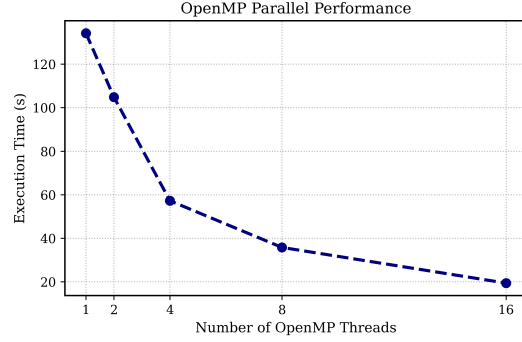


Figure 2: Execution time variation with different numbers of OpenMP threads.

Figure 2 shows that increasing the number of OpenMP threads improves performance by reducing the execution time. The improvement is due to the distribution of prediction, error computation, and gradient accumulation operations across multiple CPU threads. However, the reduction in execution time is not perfectly linear because of thread synchronization overhead, memory bandwidth limitations, and cache contention.

The accuracy, precision, recall, and F1-score remained unchanged for all thread counts. This confirms that OpenMP parallelization improved computational performance without affecting the numerical correctness or prediction quality of the Logistic Regression model.

Hybrid Performance Analysis

The MPI + OpenMP hybrid implementation combines distributed-memory parallelism with shared-memory parallelism. MPI distributes the dataset across multiple processes, while OpenMP parallelizes computations within each process.

Performance analysis showed that using a moderate number of OpenMP threads provides better results compared to higher thread counts. In this work, `OMP_NUM_THREADS = 2` gave the best performance for the hybrid configuration.

This behavior occurs because increasing the number of OpenMP threads within each MPI process leads to thread oversubscription, increased synchronization overhead, and memory contention. Since MPI already distributes the workload efficiently across processes, using a small number of threads per process minimizes overhead and improves overall efficiency.

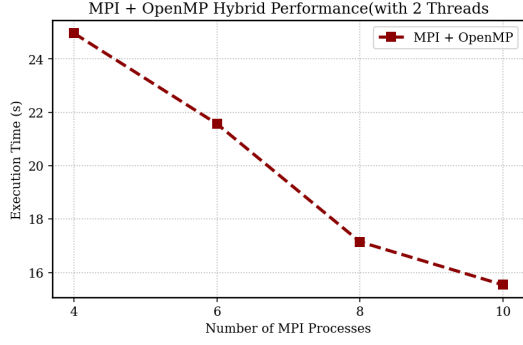


Figure 3: Execution time variation for MPI + OpenMP hybrid implementation.

The results show that increasing the number of MPI processes reduces execution time due to better workload distribution. However, excessive threading within each process does not provide additional benefits and may degrade performance.

The classification accuracy remained unchanged across all configurations, confirming that hybrid parallelization improves computational performance without affecting model correctness.

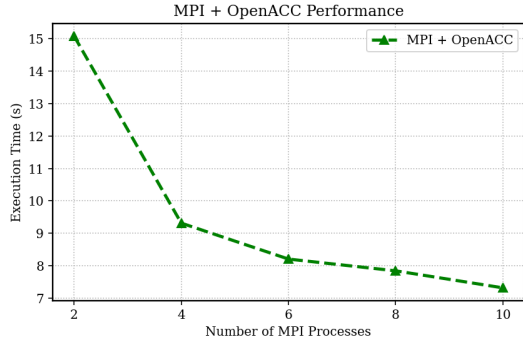


Figure 4: Execution time variation for MPI + OpenACC implementation.

The superior performance of the MPI + OpenACC implementation compared to the MPI + OpenMP hybrid approach is primarily due to GPU acceleration. GPUs are highly efficient for data-parallel operations and can execute thousands of threads simultaneously, which significantly reduces computation time for large-scale matrix and vector operations involved in Logistic Regression.

In contrast, the MPI + OpenMP implementation relies only on CPU cores, which have limited parallelism compared to GPUs. Additionally, OpenMP introduces thread synchronization and memory contention overhead, especially at higher thread counts, which reduces efficiency.

Another important factor is that OpenACC offloads the most computationally intensive portions of the code to the GPU, allowing the CPU to handle communication and coordination tasks more efficiently. This results in better utilization of system resources and improved overall performance.

The results demonstrate that combining MPI with GPU acceleration provides a highly scalable and efficient solution for large-scale machine learning workloads. The classification accuracy remained unchanged, confirming that performance improvements were achieved without affecting model correctness. For reproducibility, the plot in Figure 4 was generated using Python by varying the number of MPI processes while enabling GPU acceleration through OpenACC directives.

Performance Metrics and Model Evaluation

In this section, the performance of the proposed implementations is evaluated using key computational and classification metrics, including execution time, speedup, efficiency, accuracy, and confusion matrix analysis.

Execution Time Analysis

Execution time is used as the primary metric to evaluate computational performance. The results show a significant reduction in execution time when moving from serial to parallel implementations. The MPI + OpenACC implementation achieved the lowest execution time due to efficient GPU acceleration and distributed computation.

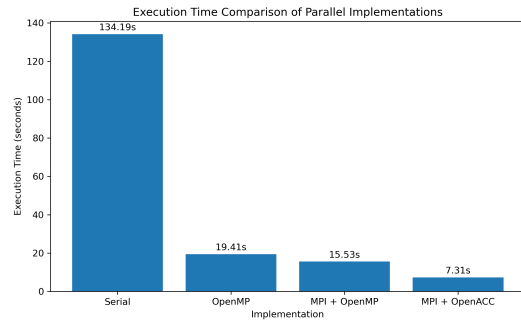


Figure 5: Execution time comparison of different implementations.

Speedup Analysis

Speedup represents the improvement in performance achieved through parallelization. It is defined as the ratio of serial execution time

to parallel execution time. The results indicate substantial speedup for OpenMP and hybrid implementations, with MPI + OpenACC showing the highest speedup.

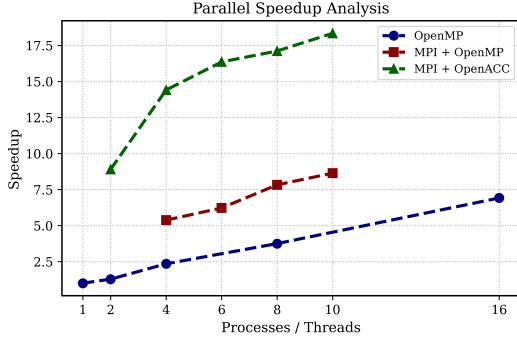


Figure 6: Speedup comparison for different parallel implementations.

Efficiency Analysis

Efficiency measures how effectively computational resources are utilized and is defined as the ratio of speedup to the number of processors. The results show that efficiency decreases with increasing parallel resources due to communication overhead and synchronization costs. However, the hybrid implementations maintain reasonable efficiency levels.

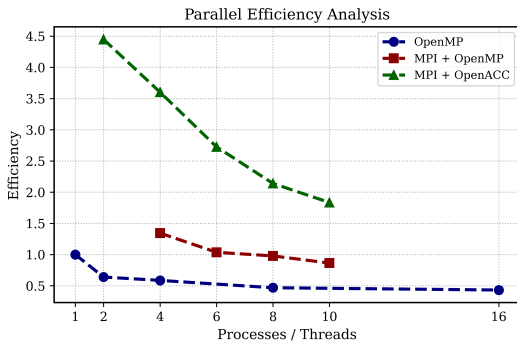


Figure 7: Parallel efficiency variation with number of processors.

Accuracy Analysis

The classification accuracy remained consistent across all implementations, with an accuracy of approximately 82.05%. This indicates that parallelization does not affect the correctness of the Logistic Regression model.

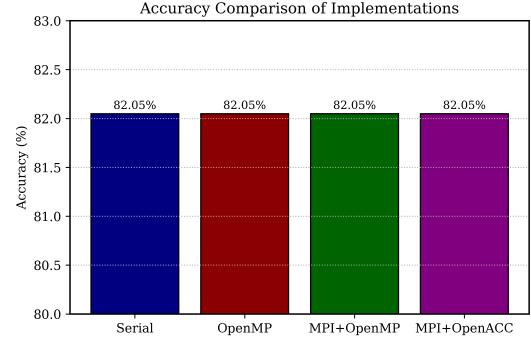


Figure 8: Accuracy comparison of serial and parallel implementations.

Confusion Matrix Analysis

The confusion matrix provides a detailed evaluation of classification performance. The model demonstrates strong recall for rainfall prediction while maintaining balanced precision.

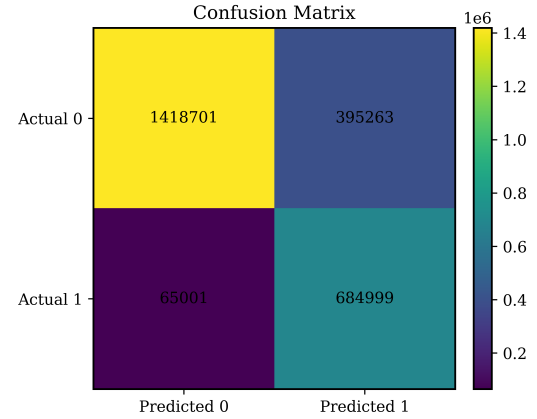


Figure 9: Confusion matrix of the Logistic Regression model.

Sample Predictions

To further evaluate the model performance, a set of randomly selected predictions from the dataset is presented in Table 1. These samples illustrate the predicted class labels along with their corresponding probabilities.

Table 1: Sample predictions from the Logistic Regression model

Actual	Predicted	Probability
0	1	0.6735
1	0	0.4660
1	1	0.9615
1	1	0.9993
0	0	0.0418
1	1	0.7444
1	1	0.9349
0	0	0.0487
1	1	0.9487
0	0	0.0086

Conclusion

The following conclusions are drawn from the present study:

- A Logistic Regression model was successfully implemented for rainfall prediction using a large-scale weather dataset containing approximately 2.56 million samples and six meteorological features.
- The serial implementation established a baseline execution time of approximately 134 seconds, highlighting the high computational cost of large-scale machine learning training.
- OpenMP-based parallelization significantly reduced execution time by utilizing multi-core CPU architecture, achieving effective shared-memory speedup.
- The MPI + OpenMP hybrid approach further improved performance by combining distributed-memory and shared-memory parallelism, enabling better workload distribution across processes and threads.
- The MPI + OpenACC implementation achieved the best performance due to GPU acceleration, demonstrating the effectiveness of offloading computationally intensive operations to the GPU.
- The optimal hybrid configuration was observed with a moderate number of OpenMP threads (2 threads per process), as higher thread counts introduced synchronization overhead and resource contention.
- Speedup increased with the number of parallel resources, while efficiency decreased gradually due to communication

overhead, memory limitations, and load imbalance.

- The classification accuracy remained approximately constant (82.05%) across all implementations, confirming that parallelization improved computational performance without affecting model correctness.
- Confusion matrix analysis showed strong recall performance for rainfall prediction, indicating the model’s ability to correctly identify rain events.
- Overall, the study demonstrates that hybrid parallel computing combined with GPU acceleration is highly effective for accelerating large-scale machine learning applications.

References

1. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. I. Foster, *Designing and Building Parallel Programs*, Addison-Wesley, 1995.
4. M. J. Quinn, *Parallel Programming in C with MPI and OpenMP*, McGraw-Hill, 2004.
5. OpenMP Architecture Review Board, “OpenMP Application Programming Interface,” Version 5.0, 2018.
6. MPI Forum, “MPI: A Message-Passing Interface Standard,” Version 3.1, 2015.
7. NVIDIA Corporation, “OpenACC Programming and Best Practices Guide,” 2023.
8. NVIDIA Corporation, “CUDA C Programming Guide,” 2024.
9. J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
10. Kaggle, “Weather Dataset,” [Online]. Available: <https://www.kaggle.com/>
11. Government of India, “India Meteorological Department (IMD),” [Online]. Available: <https://mausam.imd.gov.in/>